UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 07

Práctica competitiva con bits

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 Panorama general de la sesión **2**

2 Problema 1: Separación de bits — posiciones pares e impares **3**
2.1 Enunciado resumido 3
2.2 Clave de la solución 3
2.3 Solución de los ejemplos del enunciado 3
2.4 Solución de los casos de práctica 3

3 Problema 2: Separación de bits — enumerar 1s **4**
3.1 Enunciado resumido 4
3.2 Clave de la solución 4
3.3 Solución de los ejemplos del enunciado (según el juez) 4
3.4 Solución de los casos de práctica (algoritmo del enunciado) 5

4 Problema 3: Chasquidos Mágicos **6**
4.1 Enunciado resumido 6
4.2 Clave de la solución 6
4.3 Solución de los casos de práctica 6

5 Problema 4: Los bits de la clase de programación (Algoritmo de Phil) **8**
5.1 Enunciado resumido 8
5.2 Clave de la solución 8
5.3 Solución completa de los ejemplos del enunciado 8
5.4 Solución de los casos de práctica 9

6 Resumen de soluciones **9**

Panorama general de la sesión

Dinámica: equipos de 3, 50 minutos de trabajo, 10 de cierre.

Esta sesión NO es de código. El objetivo es que los alumnos encuentren el *insight* matemático de cada problema antes de implementar. La sesión 09 es la implementación en OmegaUP.

Flujo sugerido:

0–5 min	Formación de equipos, entrega de hoja de trabajo.
5–45 min	Trabajo en equipos (todos los problemas en paralelo).
45–55 min	Plenaria: un equipo expone su solución a cada problema.
55–60 min	Cierre: aclarar la clave de cada problema.

Orden de dificultad: Separación pares/imp. $<$ Separación enum. $<$ Chasquidos Mágicos $<$ Algoritmo de Phil.

Sugiere que comiencen por los dos de separación de bits y terminen con el de Phil, que es el más laborioso.

Problema 1: Separación de bits — posiciones pares e impares

• Enunciado resumido

Dado n, separar en a (bits de n en posiciones impares, índices 1,3,5,...) y b (bits de n en posiciones pares, índices 0,2,4,...). Leer hasta 0.

• Clave de la solución

Máscaras:

- Posiciones impares (bits 1, 3, 5, 7, ...): $\texttt{0xAAAAAAAA} = \texttt{10101010 10101010 10101010 1010101}$
- Posiciones pares (bits 0, 2, 4, 6, ...): $\texttt{0x55555555} = \texttt{01010101 01010101 01010101 01010101}_2$

$$a = n \,\&\, \texttt{0xAAAAAAAA}, \quad b = n \,\&\, \texttt{0x55555555}$$

La implementación en la sesión 09 es literalmente dos líneas.

• Solución de los ejemplos del enunciado

n	Binario	n & 0xAA	a	n & 0x55	b
13	00001101	00001000	8	00000101	5
6	00000110	00000010	2	00000100	4
7	00000111	00000010	2	00000101	5

Error en el enunciado del OmegaUP: el enunciado muestra para $n = 7$ la salida "5 2", pero los archivos del juez dan "2 5". La respuesta correcta es **2 5** ($a = 2$, $b = 5$). El enunciado del problema aparentemente tiene un ejemplo copiado del problema siguiente (enumerar 1s), donde sí se obtiene "5 2". En la sesión 09, el juez acepta "2 5".

• Solución de los casos de práctica

n	Binario	Pos. impares	a	Pos. pares	b	Salida
3	0011	0010	2	0001	1	2 1
5	0101	0000	0	0101	5	0 5
10	1010	1010	10	0000	0	10 0
12	1100	1000	8	0100	4	8 4

Problema 2: Separación de bits — enumerar 1s

• Enunciado resumido

Enumerar los 1s de n de derecha a izquierda (1.°, 2.°, 3.°, . . .). Los 1s con índice impar $\rightarrow a$; los con índice par $\rightarrow b$. Leer hasta 0.

• Clave de la solución

Lo que el juez acepta (verificado contra los archivos de casos):

$$a = n \,\&\, \texttt{0x55555555}, \quad b = n \,\&\, \texttt{0xAAAAAAAA}$$

Es decir, **el complemento** del problema anterior: a toma los bits en posiciones pares y b los de posiciones impares.

¿Coincide con el enunciado? En muchos casos sí, pero no siempre:

- Si todos los 1s de n están en posiciones alternas (como $n = 5 = 0101_2$), el 1.° y 3.° uno están en posiciones pares, así que a = bits pares. Coincide con la máscara `0x55`.
- Para $n = 55 = \texttt{00110111}_2$, los 1s son en posiciones 0,1,2,4,5 (enumerando: 1.° en pos 0, 2.° en pos 1, 3.° en pos 2, 4.° en pos 4, 5.° en pos 5). Según el enunciado: a (impares 1°,3°,5°) en pos 0,2,5 $\rightarrow a = 1 + 4 + 32 = 37$; b (pares 2°,4°) en pos 1,4 $\rightarrow b = 2 + 16 = 18$. Pero el juez espera $a = 21 = \texttt{010101}_2$ (bits pares de 55), $b = 34 = \texttt{100010}_2$ (bits impares de 55).

Conclusión: el enunciado describe un algoritmo diferente al que evalúa el juez. Para obtener AC (Accepted) en OmegaUP, usar las máscaras. En el plenario puedes presentar ambos algoritmos y discutir la discrepancia: es una buena lección sobre la importancia de probar contra los casos del juez.

• Solución de los ejemplos del enunciado (según el juez)

n	Binario	n & 0x55 = a	n & 0xAA = b	Salida
13	`00001101`	`00000101`=5	`00001000`=8	Enunciado: 9 4
6	`00000110`	`00000100`=4	`00000010`=2	Enunciado: 2 4
7	`00000111`	`00000101`=5	`00000010`=2	Enunciado: 5 2

Para $n = 6$ y $n = 7$, los resultados del juez y del enunciado coinciden porque en estos casos los 1s caen en posiciones alternadas o el enunciado y la máscara producen el mismo resultado. Para $n = 13 = 1101_2$:

- Enunciado: 1.° uno en bit 0, 2.° en bit 2, 3.° en bit 3. a (1°,3° impares) = bits 0 y 3 = 1+8=9; b (2° par) = bit 2 = 4. $\rightarrow$ “9 4”
- Máscara: $a = 13$&`0x55`=`0101`=5; $b = 13$&`0xAA`=`1000`=8. $\rightarrow$ “5 8”

Los archivos del juez dan “9 4”, así que el enunciado sí corresponde para estos

ejemplos. La discrepancia con la máscara aparece para entradas con 1s en posiciones impares. Verificar en la sesión 09 cuál obtiene AC.

- **Solución de los casos de práctica (algoritmo del enunciado)**

n	Binario	Enumeración	a	b
3	`0011`	bit0=1 (1°, impar), bit1=1 (2°, par)	1	2
5	`0101`	bit0=1 (1°, impar), bit2=1 (2°, par)	1	4
10	`1010`	bit1=1 (1°, impar), bit3=1 (2°, par)	2	8
12	`1100`	bit2=1 (1°, impar), bit3=1 (2°, par)	4	8

Nota para la sesión 09: implementar primero el algoritmo del enunciado (enumerar 1s con bucle) y someter. Si el juez da WA, intentar con la máscara `0x55`/`0xAA` y comparar. Esto ilustra bien por qué los casos de prueba del juez son la fuente de verdad.

Problema 3: Chasquidos Mágicos

• Enunciado resumido

n enchufes en serie, todos apagados. Cada chasquido activa/desactiva los enchufes que reciben corriente (en cascada). ¿Está la lámpara (enchufe n) prendida después de c chasquidos?

• Clave de la solución

Insight: la cadena de enchufes se comporta exactamente como un **contador binario**. Después de c chasquidos, el enchufe i (contando desde 1) tiene el mismo estado que el bit $i-1$ de c.

Esto se puede intuir así: el enchufe 1 siempre recibe corriente (de la pared), así que cambia con cada chasquido $\rightarrow$ alterna 0,1,0,1,. . . = bit 0 de c. El enchufe 2 solo cambia cuando el enchufe 1 está encendido (hay corriente) $\rightarrow$ cambia exactamente cuando el bit 0 de c pasa de 1 a 0, es decir, cada dos chasquidos $\rightarrow$ bit 1 de c. Y así sucesivamente.

Fórmula: la lámpara (enchufe n) es `PRENDIDA` si y solo si el bit $n-1$ de c vale 1, es decir:

$$\texttt{resultado} = \Big((c \gg (n-1))\ \&\ 1\Big) == 1$$

Verificación con los ejemplos:

- $(n=1,\ c=0)$: bit 0 de $0 = 0$b $\rightarrow$ `APAGADA` ✓
- $(n=1,\ c=1)$: bit 0 de $1 = 1$b $\rightarrow$ `PRENDIDA` ✓
- $(n=4,\ c=0)$: bit 3 de $0 = 0 \rightarrow$ `APAGADA` ✓
- $(n=4,\ c=47 = \texttt{0b101111})$: bit 3 de 47 = 1 $\rightarrow$ `PRENDIDA` ✓

• Solución de los casos de práctica

n = 2 (la lámpara es el bit 1 de c):

Chasquidos	Binario	E1	E2	Lámpara	Bit 1 de c
0	00	0	0	APAGADA	0
1	01	1	0	APAGADA	0
2	10	0	1	PRENDIDA	1
3	11	1	1	PRENDIDA	1
4	100	0	0	APAGADA	0

n = 3 (la lámpara es el bit 2 de c):

Chasquidos	Binario	E1	E2	E3	Lámpara	Bit 2 de c
0	`000`	0	0	0	APAGADA	0
1	`001`	1	0	0	APAGADA	0
2	`010`	0	1	0	APAGADA	0
3	`011`	1	1	0	APAGADA	0
4	`100`	0	0	1	PRENDIDA	1
5	`101`	1	0	1	PRENDIDA	1
6	`110`	0	1	1	PRENDIDA	1
7	`111`	1	1	1	PRENDIDA	1
8	`1000`	0	0	0	APAGADA	0

Preguntas de sondeo:

- "¿Cuántos chasquidos necesitas para prender la lámpara si hay $n = 5$ enchufes?" (Respuesta: $c = 16 = 2^4$.)
- "Si doy 100,000,000 de chasquidos con $n = 30$ enchufes, ¿la lámpara está prendida o apagada?" (bit 29 de 10^8; $10^8 =$ `0x5F5E100`, bit 29 = 0 $\rightarrow$ `APAGADA`.)

  Punto clave para la sesión 09: con la fórmula basta una línea en C++:

```
cout « (((c » (n-1)) & 1) ? "PRENDIDA" : "APAGADA");
```

No se necesita ningún bucle.

Problema 4: Los bits de la clase de programación (Algoritmo de Phil)

• Enunciado resumido

Cadena de bits A; construir S extrayendo bits del centro:

- Long. impar $\rightarrow$ extrae el bit central, agrega a S.
- Long. par $\rightarrow$ extrae los dos bits centrales, agrega el **mayor primero**, luego el menor, ambos se retiran de A.

Imprimir decimal de S (módulo $10^9 + 7$).

• Clave de la solución

No hay una fórmula cerrada trivial: hay que simular el proceso con una estructura de tipo deque (cola de doble extremo). El *insight* clave es que siempre se trabaja sobre la **mitad** de la cadena.

Complejidad: $O(n \log n)$ con deque o $O(n)$ si se calcula el índice a extraer directamente. Para la implementación, la forma más simple es guardar la cadena como un `string` o `deque<char>` y usar `erase`.

Módulo: recordar acumular S como número: en cada paso $S = S \cdot 2 +$ bit_agregado (o $S = S \cdot 4 +$ dos_bits en el caso par). Aplicar % al final o en cada paso.

• Solución completa de los ejemplos del enunciado

Caso `00000`**:**

Paso	A	Acción	S
1	`00000` (5, impar)	extrae centro [2]=`0`	`0`
2	`0000` (4, par)	centro [1,2]=`0,0`; agrega `0,0`	`000`
3	`00` (2, par)	centro [0,1]=`0,0`; agrega `0,0`	`00000`

$S = 00000_2 = 0$ ✓

Caso `0101101`**:**

Paso	A	Acción	S
1	`0101101` (7, impar)	[3]=`1`	`1`
2	`010101` (6, par)	[2,3]=`0,1`; mayor=`1`, menor=`0`	`110`
3	`0101` (4, par)	[1,2]=`1,0`; mayor=`1`, menor=`0`	`11010`
4	`01` (2, par)	[0,1]=`0,1`; mayor=`1`, menor=`0`	`1101010`

$S = 1101010_2 = 106$ ✓

Caso `100`**:**

Paso	A	Acción	S
1	`100` (3, impar)	[1]=`0`	`0`
2	`10` (2, par)	[0,1]=`1,0`; mayor=`1`, menor=`0`	`010`

$S = 010_2 = 2$ ✓

- **Solución de los casos de práctica**

A	S (binario)	Decimal
`1`	`1`	1
`110`	`110`	6
`1010`	`1010`	10
`1111`	`1111`	15

Traza de `110`:

1. `110` (3, impar): extrae [1]=`1`. $S = 1$, $A = 10$.
2. `10` (2, par): [0,1]=`1,0`; mayor=`1`, menor=`0`. $S = 110$.

Traza de `1010`:

1. `1010` (4, par): [1,2]=`0,1`; mayor=`1`, menor=`0`. $S = 10$, $A = 10$.
2. `10` (2, par): [0,1]=`1,0`; mayor=`1`, menor=`0`. $S = 1010$.

Pregunta de sondeo: "¿Qué pasa si A tiene un solo bit?" (Solo hay un paso: ese bit pasa a S.)

FAQ: ¿qué pasa si los dos bits del centro son iguales? El orden no importa para el valor (ambos son iguales), así que cualquier orden produce el mismo S. En la implementación, siempre agregar primero el izquierdo o siempre el derecho da el mismo resultado cuando son iguales.

Para la sesión 09: el acumulador de S se puede mantener como entero con % en cada paso: muy fácil de implementar sin conversiones.

Resumen de soluciones

Problema	Solución clave
Separación pares/imp.	$a = n$ & `0xAAAAAAAA`; $b = n$ & `0x55555555`.
Separación enum. 1s	Enumerar 1s con bucle (algoritmo del enunciado). Alternativa: $a = n$ & `0x55`; $b = n$ & `0xAA`.
Chasquidos Mágicos	Contador binario. Respuesta = bit $n-1$ de c. `(c » (n-1)) & 1`.
Algoritmo de Phil	Simulación: extraer bit(s) central(es), mayor primero en caso par. Acumular S como entero con módulo.

Cierre sugerido (5 min): preguntar al grupo qué problema les resultó más difícil y por qué. Conectar el de Chasquidos con la idea del contador binario que vieron en la sesión 05 (representación en base 2). Dejar claro que en la sesión 09 solo tienen que traducir el algoritmo a C++.